Bonus Lecture: Time vs Space for RAMs

Guess-and-verify gates and Cook–Mertz

In Lecture 6, we saw how Williams's theorem follows by converting a time-T multitape Turing-machine computation into a circuit with large fan-in and small depth, and then using Cook–Mertz to evaluate the circuit in much less space than ordinary recursion would require. Specifically, a depth-d circuit with fan-in w and locally computable gate operations can be evaluated in space $\widetilde{O}(w + d)$ rather than the naive $\widetilde{O}(wd)$. The important idea is that each recursive level borrows the *same* global workspace.

For RAM computations there is no analogous locality of memory addresses. Nevertheless, a suitable modification of the circuit model lets us prove the following.

Theorem 1 (RAM time versus space)**.** *A deterministic RAM computation of time T can be simulated deterministically in space*

$$\boxed{\widetilde{O}(T^{2/3})}.$$

Here the RAM has a fixed number of $O(\log T)$-bit processor registers, $O(\log T)$-bit addresses and memory words, and standard local operations. Simulation time is unrestricted.

The argument has three parts, in this order:

1. Define circuits with *guess-and-verify* gates and a weighted notion of depth.
2. Extend the Cook–Mertz evaluator to these circuits.
3. Describe the RAM computation by such a shallow, narrow circuit.

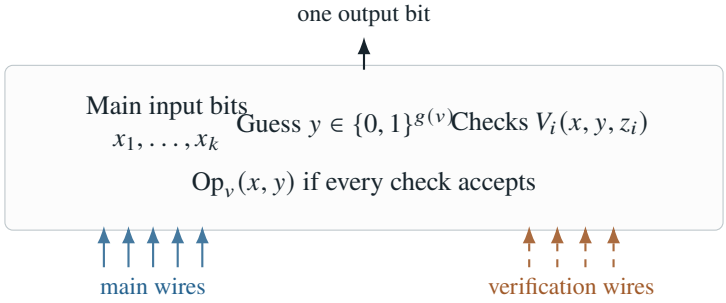
We will ignore logarithmic factors throughout, writing $\widetilde{O}$ for $O(\cdot\ \mathrm{polylog}\, T)$.

1 Guess-and-verify gates

Fix a fan-in parameter w. All wires carry individual bits.

Definition 2 (Guess-and-verify gate)**.** A gate v has $k \le w$ *main input bits* $x = (x_1, \dots, x_k)$, and any number of *verification input bits* $z_1, \dots, z_m$. It also has a guess length $g(v) \le w$, an output function $\mathrm{Op}_v(x, y)$, and verification functions $V_{v,i}(x, y, z_i)$ for $1 \le i \le m$, where $y \in \{0,1\}^{g(v)}$.

Requirement: for every Boolean assignment of $x, z_1, \dots, z_m$, there is exactly one guess y for which $V_{v,i}(x, y, z_i) = 1$ for all i. The gate outputs $\mathrm{Op}_v(x, y)$ for that unique guess. The functions Op_v and $V_{v,i}$ are computable in $\widetilde{O}(w)$ space when their inputs are given explicitly.



Despite the guessing, this is a *deterministic* operation: we can enumerate y, check the verification bits one at a time, and return $\mathrm{Op}_v(x, y)$ for the unique successful guess. When its input bits are explicitly available, this uses $\widetilde{O}(w)$ space.

For the moment a circuit may have arbitrary fan-out; we will return to this after proving the evaluator. A zero-guess gate ($g(v) = 0$) is an ordinary deterministic gate, with its input bits taken as main inputs and no verification inputs. Thus no separate kind of non-input gate is needed.

Depth and verification depth

Write d for ordinary circuit depth: the maximum number of gates along a dependency path, running backward from the output to an input. Along such a path, entering a gate v through a main input has *weight zero*; entering it through a verification input has weight $g(v)$. Define its *verification depth*

$$d_v = \max_{\text{dependency paths } \pi} \sum_{\substack{v \in \pi:\ \pi \text{ uses a verification} \\ \text{input of } v}} g(v). \tag{1}$$

The naive recursive evaluator saves up to w main input bits at each suspended gate, plus the guess y when descending through a verification input. Its space usage is therefore

$$\boxed{\widetilde{O}(wd + d_v).}$$

What we want is the Cook–Mertz replacement $wd \rightsquigarrow w + d$, without changing the d_v term.

Theorem 3 (Cook–Mertz for guess-and-verify circuits)**.** *A polynomial-size, uniformly accessible guess-and-verify circuit of fan-in w, depth d, and verification depth d_v can be evaluated deterministically in*

$$\boxed{\widetilde{O}(w + d + d_v)} \quad \textit{space}.$$

2 Arithmetizing a gate

As in Lecture 6, let p be a prime of polynomial magnitude and work over $\mathbb{F}_p$. Call its elements *bytes*; one byte occupies $O(\log T)$ actual bits. We choose p sufficiently large for the interpolation degrees below, and find it in logarithmic space by trial division.

Any Boolean function $f : \{0,1\}^k \to \{0,1\}$ has a multilinear polynomial extension

$$\widehat{f}(X) = \sum_{a \in \{0,1\}^k} f(a) \prod_{j : a_j = 1} X_j \prod_{j : a_j = 0} (1 - X_j). \tag{2}$$

It has degree at most k, agrees with f on Boolean inputs, and is evaluable at arbitrary byte inputs using $\widetilde{O}(w)$ space: enumerate the Boolean a's and accumulate one field sum. The local evaluation makes no recursive calls to other circuit gates.

Fix a guess-and-verify gate v. For every Boolean y, extend $\mathrm{Op}_v(x, y)$ multilinearly in x, and similarly extend each $V_{v,i}(x, y, z_i)$ multilinearly in x (with y, z_i kept Boolean). Then form

$$F_v(X; z_1, \ldots, z_m) = \sum_{y \in \{0,1\}^{g(v)}} \widehat{\mathrm{Op}}_v(X, y) \prod_{i=1}^{m} \widehat{V}_{v,i}(X, y, z_i). \tag{3}$$

For the actual Boolean main and verification inputs, exactly one y contributes, and therefore

$$F_v(x; z_1, \ldots, z_m) = \mathrm{bit}(v). \tag{4}$$

For fixed Boolean $z_1, \ldots, z_m$, the degree of F_v in its main inputs X is at most

$$D_v = (m+1)w.$$

(The number $2^{g(v)}$ of guesses affects time, but not polynomial degree.) Let D bound all D_v and the ordinary gate degrees. The number of verification inputs is polynomial in the circuit size, so $D = T^{O(1)}$. We choose $p > D + 1$; thus p has $O(\log T)$ bits and admits $D + 1$ distinct nonzero interpolation points.

The same thorny situation as Lecture 6

Suppose the actual main input bits are $B = (B_1, \ldots, B_k)$, but our global byte registers initially contain unknown values $A = (A_1, \ldots, A_k)$. We will be able to evaluate

$$q(t) = F_v(A_1 t + B_1, \ldots, A_k t + B_k; z),$$

for chosen nonzero t. The task is to recover $q(0) = F_v(B; z)$.

If $\deg q \le D_v$, choose $D_v + 1$ distinct nonzero field elements $t_1, \ldots, t_{D_v+1}$. Their Lagrange coefficients at 0 are

$$\mu_j = \prod_{\substack{1 \le r \le D_v+1 \\ r \ne j}} \frac{-t_r}{t_j - t_r}, \qquad q(0) = \sum_{j=1}^{D_v+1} \mu_j q(t_j). \tag{*}$$

The points and coefficients can be enumerated and recomputed using $O(\log T)$ space. Nothing about this interpolation differs from Lecture 6. The only new issue is that evaluating $q(t)$ requires querying many verification-input bits recursively.

3 The Cook–Mertz evaluator

The global bytes and the inductive guarantee

Keep exactly $w + 2$ global byte registers,

$$\boxed{\mathrm{Byte}_1, \mathrm{Byte}_2, \ldots, \mathrm{Byte}_{w+2}.}$$

They are shared by *all* recursive calls. We prove by induction on the circuit that for any gate v, any register r, and either sign $\varepsilon \in \{+1, -1\}$, there is a subroutine

$$\mathsf{ADD}(v, \varepsilon, r): \qquad \mathrm{Byte}_r \longleftarrow \mathrm{Byte}_r + \varepsilon\, \mathrm{bit}(v),$$

which leaves *every other global byte unchanged*. This must hold from arbitrary initial contents of the entire register pool. Input gates are immediate. Subtraction is constructed simultaneously with addition.

We explain the routine for Byte_1 as its target. We give three names to its registers and subroutines:

- $X = (\mathrm{Byte}_2, \ldots, \mathrm{Byte}_{k+1})$, the $k \le w$ main-input registers; these do *not* initially contain the main input bits.
- $Z = \mathrm{Byte}_{w+2}$, one spare global register for querying a verification bit. It too has arbitrary initial contents.
- $\mathsf{READ}(u)$ returns the actual Boolean output of a gate u while restoring every global register. $\mathsf{EVAL}_v(X)$ returns $F_v(X; z)$, also restoring every global register.

For another target we merely choose and label distinct registers afresh, without moving their contents. There are always enough: w main-input slots and one spare slot besides the target.

3.1 Reading one verification bit

The output of a verification-input gate u is a single bit $z = \mathrm{bit}(u)$. To obtain it we need only the spare byte Z. Locally save one byte, and use the inductive clean-addition subroutine:

```
READ(u):
    old = Z
    ADD(u, +1, Z)           // Z is now old + bit(u)
    z = Z - old             // z is the actual bit(u)
    ADD(u, -1, Z)           // restores Z
    return z
```

Guarantee: $\mathsf{READ}(u)$ returns $\mathrm{bit}(u)$ and restores *all* global bytes, including Z and X. The local variables `old` and `z` occupy only two bytes on the usual recursion stack. No special polynomial identity is needed for the verification bits.

3.2 Evaluating a gate polynomial

Suppose the current contents of the main-input registers are the field vector X, possibly with non-Boolean entries. We want the number $F_v(X; z_1, \ldots, z_m)$ from (3).

```
EVAL_v(X):
    sum = 0
    for y in {0,1}^{g(v)}:
        product = Op_hat_v(X,y)
        for i = 1,...,m:
            z = READ(u_i)              // z = actual verification bit
            product *= V_hat_{v,i}(X,y,z)
        sum += product
    return sum
```

Guarantee: $\mathsf{EVAL}_v(X)$ returns $F_v(X; z_1, \ldots, z_m)$ and leaves *all* global registers unchanged. It stores just the current $g(v)$-bit guess, two scalar accumulators, indices, and small bookkeeping. The local polynomial lookups use one reusable $\widetilde{O}(w)$-bit area, which is released before the next READ call.

Notice the position of the recursive calls: EVAL_v queries *verification inputs* while holding y. It does not query any of v's main inputs.

3.3 Adding the gate's output

Let $u_1, \ldots, u_k$ be the main-input gates of v, with actual outputs $B_1, \ldots, B_k$. Suppose X initially contains an arbitrary unknown vector A. To implement $\mathsf{ADD}(v, \varepsilon, 1)$, use exactly the interpolation loop of Lecture 6:

```
ADD(v, eps, Byte_1):
    for j = 1,...,D_v+1:
        t = t_j
        X *= t
        for r = 1,...,k:
            ADD(u_r, +1, X_r)           // X = tA + B

        Byte_1 += eps * mu_j * EVAL_v(X)

        for r = k,...,1:
            ADD(u_r, -1, X_r)           // X = tA
        X /= t                          // X = A again
```

Every recursive main-input call is clean by induction. Thus during the middle line $X = t_j A + B$, and $\mathsf{EVAL}_v(X)$ returns $q(t_j)$ while restoring every global register. The reverse calls and division restore X to A. Summing the target increments over j and using (*) gives

$$\Delta\mathrm{Byte}_1 = \varepsilon \sum_{j=1}^{D_v+1} \mu_j q(t_j) = \varepsilon q(0) = \varepsilon F_v(B; z) = \varepsilon\,\mathrm{bit}(v).$$

This proves the inductive guarantee: the desired target changed, and all other global bytes returned to their original values.

When $g(v) = 0$ and $m = 0$, $\mathsf{EVAL}_v(X)$ is just a locally computed multilinear gate operation. Thus the *same procedure* handles ordinary gates; we have not introduced a second type of gate.

3.4 Where the space goes

There are three costs, and only three:

1. **Global workspace: $\widetilde{O}(w)$.** The $w + 2$ byte registers, a description of $\mathbb{F}_p$, and one reusable local lookup area. All descendants use the same workspace. A local lookup releases its temporary data before invoking a recursive gate.
2. **Ordinary recursion stack: $\widetilde{O}(d)$.** Each suspended call stores the gate identifier, its current interpolation point, its target index, a few bytes, and short loop indices. There are at most d nested gate evaluations.
3. **Saved guesses: $O(d_v)$.** When we recursively follow a *main* input via $\mathsf{ADD}(u_r, \pm 1, X_r)$, this happens outside EVAL_v and no guess for v is retained. When we follow a *verification* input via $\mathsf{READ}(u_i)$, we are inside the loop over y, and must retain this $g(v)$-bit guess until the child returns.

At every instant the nested recursive calls follow a dependency path. The sum of the saved guess lengths on that path is at most d_v by definition. A guess belonging to the currently active (unsuspended) gate has length at most w and fits in the global/local workspace term. Thus the total space is

$$\boxed{\widetilde{O}(w + d + d_v).}$$

This completes the proof of Theorem 3. Initializing one target byte to zero and calling ADD on the output gate computes the answer.

4 Multiple fan-out

Nothing in the induction assumes that a gate is used only once. We proved the clean-addition routine for arbitrary circuit gates, even when their output feeds many later gates. The evaluator simply recomputes such a gate whenever needed. This can increase running time dramatically, but the active recursive calls still form one dependency path. Consequently the same theorem holds for *DAGs*, with unrestricted fan-out.

A word-valued operation returning r bits can also be implemented by r separate one-bit gates in parallel, all with the same main and verification inputs and the same guess. This increases the number of gates but does not multiply circuit depth or verification depth: a dependency path encounters at most one of these parallel output gates per layer.

5 Simulating RAM time by guess-and-verify circuits

5.1 The two tableaus

We use a *bit-RAM* normal form. Its memory has $N = T^{O(1)}$ individually addressable bits $\mathrm{MEM}[i]$. The processor has $h = O(\log T)$ bits of state (program counter, clock, address and a fixed number of registers). In each microstep, the processor state determines the address i to read; the next state and any write to $\mathrm{MEM}[i]$ depend only on that state and the bit read. Processor-only actions can use a dummy address. A word-RAM instruction can be simulated by a polylogarithmic number of these bit microsteps, without changing the exponent in our space bound.

Think of a computation tableau with time going downward. Its narrow side records the processor state after every microstep. Its wide side records all the memory bits, of which only one is accessed at each step. The construction will *not* store the wide tableau.

Choose a circuit fan-in w and divide time into epochs of $E = \Theta(w/h) = \widetilde{\Theta}(w)$ steps. Choose a chunk length $c \le E$. Define

$$\mathrm{STATE}_e[0..t] = \text{the processor-state history through step } t \text{ of epoch } e,$$
$$\mathrm{MEM}_e[i] = \text{memory bit } i \text{ at the beginning of epoch } e.$$

Each state record includes the bit read on its step, so the list of records determines all writes. Choose the constant in $E = \Theta(w/h)$ so that the entire epoch history, together with one additional bit, has length at most w.

5.2 One chunk is a parallel family of guess-and-verify gates

Suppose $\mathrm{STATE}_e[0..t]$ is known as a tuple of wires. To generate the next c processor states, use the previous history as the *main inputs*, and use *all* the epoch-start memory bits $\mathrm{MEM}_e[i]$ as separate verification inputs.

A guess $y \in \{0,1\}^c$ supplies the read bits for the next c steps. From the current processor state and these guessed bits, the next c processor states and their writes are computable without consulting memory. This is the proposed output.

For each address i, the verification predicate checks the guesses for reads from i: starting with $\mathrm{MEM}_e[i]$, first apply all writes to i recorded in the *previous* epoch history; then follow the proposed chunk and check each guessed read at i, applying each new write to its one-bit working buffer. Crucially, the checker does *not* demand that earlier recorded reads were consistent. It treats the old history as authoritative. Hence, for every Boolean main-history input and every assignment of the epoch-start memory bits, there is exactly one consistent c-bit guess: the real reads of the next c steps starting from those inputs. This verifies the uniqueness condition of Definition 1, including on arbitrary Boolean gate inputs.

The c new processor states contain $O(c \log T)$ bits. We obtain them from *parallel* one-bit guess-and-verify gates, each with the same inputs and the same c-bit guess but with a different output coordinate. Their depth contribution is one layer, not $O(c)$. Previously produced history bits are simply retained as wires.

5.3 The epoch boundary

At the end of epoch e, each memory bit $\mathrm{MEM}_{e+1}[i]$ is computed from exactly $\mathrm{STATE}_e[0..E]$ and $\mathrm{MEM}_e[i]$: scan the complete epoch history for the last write to i, using its written bit if present, and otherwise copying $\mathrm{MEM}_e[i]$. This is a *zero-guess gate* with at most w main input bits (the history and one memory bit). The last processor state of epoch e becomes the initial processor state of epoch $e+1$ by ordinary wires.

There are N separate memory-update gates at each boundary, but their values are never stored together. The complete circuit has polynomially many gates and wire occurrences. Each gate is addressed by an epoch, chunk, coordinate, and perhaps a memory address; its predecessors and local operation can be found with polylogarithmic-space counters. In particular, the circuit itself is not materialized by the space-bounded simulator.

5.4 The two path parameters

There are $O(T/E)$ epochs, and $O(E/c)$ chunk layers per epoch. Epoch boundaries contribute only constant depth, so

$$d = O\left(\frac{T}{E}\left(\frac{E}{c} + 1\right)\right) = \widetilde{O}(T/c). \tag{5}$$

Every chunk gate has $g(v) = c$. A dependency path may follow many *main-input* edges between chunks in the same epoch, but these have verification weight 0. Once the path follows a *verification* edge of a chunk gate, it arrives at $\mathrm{MEM}_e[i]$, which is a bit from the *beginning* of the epoch. That bit depends only on the previous epoch. Therefore a path incurs at most one c-bit verification charge in each epoch, giving

$$d_v = O\left(\frac{T}{E}\, c\right) = \widetilde{O}(Tc/w). \tag{6}$$

This is the reason for using *both* epochs and shorter chunks: verifying a chunk against the fixed epoch-start memory makes verification-input edges jump across the earlier chunks, while limiting the guessed string to c bits.

5.5 Balance the parameters

Theorem 3, together with (5)–(6), gives

$$\text{Space} = \widetilde{O}\left(w + \frac{T}{c} + \frac{Tc}{w}\right).$$

For fixed w, balance the last two terms by choosing $c \asymp \sqrt{w}$. Then

$$\text{Space} = \widetilde{O}\left(w + \frac{T}{\sqrt{w}}\right),$$

which is balanced at $w \asymp T^{2/3}$ and $c \asymp T^{1/3}$. Hence

$$\boxed{\mathrm{RAMTIME}(T) \subseteq \mathrm{DSPACE}(\widetilde{O}(T^{2/3}))},$$

as claimed. The proof uses the same Cook–Mertz interpolation as Lecture 6; its only extra recursion cost is the verification depth d_v.

References

[1] James Cook and Ian Mertz. *Tree Evaluation Is in Space $O(\log n \log\log n)$*. STOC 2024.

[2] R. Ryan Williams. *Simulating Time with Square-Root Space*. STOC 2025.